

■ BitQuest v2

Plan de arquitectura, lógica y trabajo

3 integrantes · Windows x64 · PlaySound + subcarpetas · 4 días

Este documento cubre la arquitectura revisada de BitQuest con subcarpetas, la lógica detrás de cada módulo, las bases de código de cada archivo, la integración de música con PlaySound/winmm, y el plan de 4 días ajustado a la nueva distribución de integrantes.

Integrante	Archivos propios	Compartido
Int. 1 — Jugador y Renderizado	src/jugador.c · include/jugador.h src/renderizado.c · include/renderizado.h	main.c · assets.h · colores.h
Int. 2 — NASM y Build	src/rutinas.asm · include/rutinas.h build.bat · README.md	main.c · assets.h · colores.h
Int. 3 — Mapas, Objetos y Audio	src/mapas.c · include/mapas.h src/objetos.c · include/objetos.h include/colores.h · include/assets.h	main.c · assets.h · colores.h
Todos — Main	src/main.c	Integración final conjunta

Estructura de carpetas del proyecto

Todos los archivos .c van en src/, los .h en include/, los mapas en mapas/ y los .wav en assets/.

```
BitQuest/  
  ■■■ src/  
  ■ ■■■ main.c ← todos  
  ■ ■■■ jugador.c ← Int. 1  
  ■ ■■■ renderizado.c ← Int. 1  
  ■ ■■■ mapas.c ← Int. 3  
  ■ ■■■ objetos.c ← Int. 3  
  ■ ■■■ rutinas.asm ← Int. 2  
  ■■■ include/  
  ■ ■■■ jugador.h ← Int. 1  
  ■ ■■■ renderizado.h ← Int. 1  
  ■ ■■■ mapas.h ← Int. 3  
  ■ ■■■ objetos.h ← Int. 3  
  ■ ■■■ rutinas.h ← Int. 2  
  ■ ■■■ colores.h ← Int. 3  
  ■ ■■■ assets.h ← Int. 3  
  ■■■ mapas/  
  ■ ■■■ nivel1.txt  
  ■ ■■■ nivel2.txt  
  ■ ■■■ nivel3.txt  
  ■ ■■■ nivel4.txt  
  ■■■ assets/  
  ■ ■■■ nivel1.wav  
  ■ ■■■ nivel2.wav  
  ■ ■■■ nivel3.wav  
  ■ ■■■ nivel4.wav  
  ■■■ build.bat ← Int. 2  
  ■■■ README.md ← Int. 2
```

Lógica general del proyecto

Estructuras de datos centrales

Todo el juego gira en torno a dos structs definidos en jugador.h y mapas.h. NASM solo recibe punteros a la matriz y opera sobre bytes — no conoce los structs.

```
// include/jugador.h
typedef struct {
int fila, col; // posicion actual en el mapa completo
int monedas; // monedas recogidas en este nivel
int tiene_llave; // 0 = no, 1 = si
int pasos; // pasos dados en este nivel
} Jugador;

// include/mapas.h
typedef struct {
char mapa[FILAS][COLS]; // 60x60 chars en memoria contigua
int total_monedas; // calculado por NASM al cargar
int total_libres; // calculado por NASM al cargar
int num_nivel;
} Nivel;
```

Acceso de NASM a la matriz

El mapa[60][60] es un bloque contiguo de 3600 bytes en RAM. Para acceder a la celda [fila][col] NASM usa la fórmula: $\text{offset} = \text{fila} * \text{COLS} + \text{col}$. En Windows x64 los parámetros llegan en RCX, RDX, R8, R9 (no RDI/RSI como en Linux).

```
; Para acceder a mapa[fila][col]:
; RCX = puntero base del mapa
; RDX = numero de columnas (60)
; R8 = fila
; R9 = col
imul r10, r8, rdx ; offset = fila * COLS
add r10, r9 ; offset += col
movzx rax, byte [rcx + r10] ; leer el caracter
```

Flujo por frame (cada tecla presionada)

1. main.c lee tecla con `_getch()`
2. Llama a `mover_jugador()` de jugador.c
3. jugador.c llama `validar_movimiento()` de NASM → si devuelve 0, termina
4. jugador.c llama `detectar_objeto()` de NASM → obtiene qué hay en la celda destino
5. objetos.c maneja el efecto: recoger moneda, llave, abrir puerta, detectar salida
6. renderizado.c calcula la cámara y dibuja la ventana 20×20 con colores ANSI
7. renderizado.c imprime el HUD (nivel, monedas, pasos, llave)

Lógica de la ventana visible 20×20

La cámara se centra en el jugador y se clampea para no salir del mapa. Esto vive completamente en renderizado.c:

```
int cam_fila = jugador->fila - VIS/2;
int cam_col = jugador->col - VIS/2;
if (cam_fila < 0) cam_fila = 0;
if (cam_col < 0) cam_col = 0;
if (cam_fila > FILAS - VIS) cam_fila = FILAS - VIS;
if (cam_col > COLS - VIS) cam_col = COLS - VIS;
// luego iterar: for i in [cam_fila, cam_fila+VIS)
// for j in [cam_col, cam_col+VIS)
```

Lógica de música con PlaySound (winmm)

PlaySound reproduce un archivo .wav en un hilo de fondo sin bloquear el juego. Se llama desde main.c al iniciar cada nivel. SND_ASYNC permite que el juego siga corriendo mientras suena la música. SND_LOOP la repite hasta que se llame PlaySound(NULL,...) para detenerla.

```
#include // en mapas.h o en main.c
#pragma comment(lib, "winmm.lib") // solo si usas MSVC; con GCC usa -lwinmm en build.bat
// Al iniciar un nivel:
PlaySound("assets/nivel1.wav", NULL, SND_FILENAME | SND_ASYNC | SND_LOOP);
// Al terminar el nivel o al salir:
PlaySound(NULL, NULL, SND_PURGE);
```

Integrante 1 — Jugador y Renderizado

Maneja todo lo relacionado con el estado del jugador y lo que se muestra en pantalla. Sus dos pares de archivos (.h + .c) son independientes entre sí y del resto.

include/jugador.h — base del archivo

```
#ifndef JUGADOR_H
#define JUGADOR_H

#include "mapas.h" // necesita Nivel y las constantes FILAS/COLS
#include "rutinas.h" // prototipos NASM: validar_movimiento, detectar_objeto
#include "objetos.h" // para que mover_jugador pueda llamar manejar_objeto
#define VIS 20 // tamaño de la ventana visible

typedef struct {
    int fila, col;
    int monedas;
    int tiene_llave;
    int pasos;
} Jugador;

void jugador_init(Jugador *j, int fila_inicio, int col_inicio);
void mover_jugador(Nivel *n, Jugador *j, char tecla);
#endif // JUGADOR_H
```

src/jugador.c — base del archivo

jugador_init() pone al jugador en su posición inicial. mover_jugador() es el núcleo: llama a NASM para validar y detectar, luego delega el efecto a objetos.c.

```
#include "../include/jugador.h"
#include "../include/objetos.h"
#include "../include/rutinas.h"
#include

void jugador_init(Jugador *j, int fila_inicio, int col_inicio) {
    j->fila = fila_inicio;
    j->col = col_inicio;
    j->monedas = 0;
    j->tiene_llave = 0;
    j->pasos = 0;
}

void mover_jugador(Nivel *n, Jugador *j, char tecla) {
    int nf = j->fila, nc = j->col;
    if (tecla=='w' || tecla=='W') nf--;
    else if (tecla=='s' || tecla=='S') nf++;
    else if (tecla=='a' || tecla=='A') nc--;
    else if (tecla=='d' || tecla=='D') nc++;
    else return;

    // NASM valida si la celda destino es transitable
    if (!validar_movimiento((char*)n->mapa, COLS, nf, nc)) return;
    // NASM detecta qué objeto hay en la celda destino
    // manejar_objeto actualiza el mapa y el estado del jugador
    if (manejar_objeto(n, j, nf, nc)) {
        j->fila = nf;
        j->col = nc;
        j->pasos++;
    }
}
```

include/renderizado.h — base del archivo

```
#ifndef RENDERIZADO_H
#define RENDERIZADO_H

#include "jugador.h"
#include "mapas.h"
#include "colores.h"
#include "assets.h"

void limpiar_pantalla(void);
void imprimir_ventana(Nivel *n, Jugador *j);
void mostrar_hud(Jugador *j, int num_nivel);
void mostrar_resumen_nivel(Jugador *j, Nivel *n);
void mostrar_resumen_final(int mon_total, int mon_max, int pasos, long puntaje);
void pantalla_inicio(void);
void pantalla_victoria(long puntaje);
#endif // RENDERIZADO_H
```

src/renderizado.c — base del archivo

imprimir_ventana() calcula la cámara, itera los 20×20 tiles y para cada char lógico imprime el VISUAL_* con su COLOR_* correspondiente.

limpiar_pantalla() usa system("cls") en Windows.

```
#include "../include/renderizado.h"
#include
#include

void limpiar_pantalla(void) { system("cls"); }

void imprimir_ventana(Nivel *n, Jugador *j) {
    limpiar_pantalla();
    int cf = j->fila - VIS/2;
    int cc = j->col - VIS/2;
    if (cf < 0) cf = 0;
    if (cc < 0) cc = 0;
    if (cf > FILAS - VIS) cf = FILAS - VIS;
    if (cc > COLS - VIS) cc = COLS - VIS;
    for (int i = cf; i < cf + VIS; i++) {
        for (int k = cc; k < cc + VIS; k++) {
            char c = (i==j->fila && k==j->col) ? CHAR_JUGADOR : n->mapa[i][k];
            switch (c) {
                case CHAR_PARED: printf(COLOR_PARED VISUAL_PARED COLOR_RESET); break;
                case CHAR_MONEDA: printf(COLOR_MONEDA VISUAL_MONEDA COLOR_RESET); break;
                case CHAR_LLAVE: printf(COLOR_LLAVE VISUAL_LLAVE COLOR_RESET); break;
                case CHAR_PUERTA: printf(COLOR_PUERTA VISUAL_PUERTA COLOR_RESET); break;
                case CHAR_SALIDA: printf(COLOR_SALIDA VISUAL_SALIDA COLOR_RESET); break;
                case CHAR_JUGADOR: printf(COLOR_JUGADOR VISUAL_JUGADOR COLOR_RESET); break;
                default: printf(VISUAL_LIBRE); break;
            }
        }
    }
    printf("\n");
}

void mostrar_hud(Jugador *j, int num_nivel) {
    printf(COLOR_TITULO
        " Nivel: %d | Monedas: %d | Pasos: %d | Llave: %s\n"
        COLOR_RESET,
        num_nivel, j->monedas, j->pasos,
        j->tiene_llave ? "Si" : "No");
}

// TODO: mostrar_resumen_nivel, mostrar_resumen_final,
```

```
// pantalla_inicio, pantalla_victoria
```

⇒ *Depende de: mapas.h (Nivel, FILAS, COLS), rutinas.h (NASM), objetos.h*

⇒ *Entrega el día 1: jugador.h con Jugador typedef y prototipos — desbloquea a todos*

Integrante 2 — NASM y Build System

Las 5 funciones NASM valen 30 puntos directos. También construye el build.bat que compila el proyecto completo con subcarpetas y winmm.

include/rutinas.h — base del archivo

src/rutinas.asm — base del archivo (las 5 funciones)

Todas usan la convención Windows x64. El parámetro del char llega como entero en R8 o R9 pero solo se compara el byte bajo. detectar objeto tiene 5 parámetros — el quinto llega en la pila ([rsp+40] tras el prólogo).

[illegible]

[illegible]

build.bat — compilación completa con subcarpetas y winmm

El build.bat ensambla rutinas.asm, compila todos los .c de src/ e incluye -I include/ para los headers y -lwinmm para PlaySound.

```
if %errorlevel% neq 0 ( echo ERROR en GCC & pause & exit /b 1 )  
echo [3/3] Listo. Ejecutando...  
BitQuest.exe
```

- ⇒ Entrega rutinas.h el día 1 — desbloquea al Int. 1 para usar stubs temporales
- ⇒ El 5to parámetro de detectar_objeto llega en [rsp+40] por convención Windows x64
- ⇒ build.bat usa ^ para continuar línea larga en CMD — no agregar espacios tras ^

Integrante 3 — Mapas, Objetos, Colores, Assets

Construye el contenido del juego: los 4 mapas, la lógica de objetos, los headers de colores y assets, y la música con PlaySound.

include/colores.h — base del archivo

```
#ifndef COLORES_H
#define COLORES_H

#define COLOR_RESET "\033[0m"
#define COLOR_PARED "\033[90m"
#define COLOR_JUGADOR "\033[93m"
#define COLOR_MONEDA "\033[33m"
#define COLOR_LLAVE "\033[96m"
#define COLOR_PUERTA "\033[31m"
#define COLOR_SALIDA "\033[92m"
#define COLOR_TITULO "\033[95m"
#define COLOR_EXITO "\033[92m"

#endif // COLORES_H
```

include/assets.h — base del archivo

```
#ifndef ASSETS_H
#define ASSETS_H

// Caracteres logicos (viven en la matriz)
#define CHAR_PARED '#'
#define CHAR_LIBRE '.'
#define CHAR_JUGADOR 'P'
#define CHAR_MONEDA 'M'
#define CHAR_LLAVE 'K'
#define CHAR_PUERTA 'D'
#define CHAR_SALIDA 'E'

// Visuales para renderizado (todos 2 caracteres)
#define VISUAL_PARED "##"
#define VISUAL_LIBRE " "
#define VISUAL_JUGADOR "@ "
#define VISUAL_MONEDA "o "
#define VISUAL_LLAVE "! "
#define VISUAL_PUERTA "[]"
#define VISUAL_SALIDA "><"

// Rutas de audio por nivel
#define AUDIO_NIVEL1 "assets/nivel1.wav"
#define AUDIO_NIVEL2 "assets/nivel2.wav"
#define AUDIO_NIVEL3 "assets/nivel3.wav"
#define AUDIO_NIVEL4 "assets/nivel4.wav"

#endif // ASSETS_H
```

include/mapas.h — base del archivo

```
#ifndef MAPAS_H
#define MAPAS_H

#define FILAS 60
#define COLS 60
#define NUM_NIVELES 4

typedef struct {
    char mapa[FILAS][COLS];
    int total_monedas;
    int total_libres;
```

```

int num_nivel;
} Nivel;

int cargar_mapa(const char *ruta, Nivel *n);
void encontrar_jugador(Nivel *n, int *fila_out, int *col_out);
#endif // MAPAS_H

```

src/mapas.c — base del archivo

cargar_mapa() lee el .txt y llena el struct Nivel. Tras cargarlo llama a las funciones NASM para calcular total_monedas y total_libres automáticamente.

```

#include "../include/mapas.h"
#include "../include/rutinas.h"
#include
#include

int cargar_mapa(const char *ruta, Nivel *n) {
FILE *f = fopen(ruta, "r");
if (!f) { fprintf(stderr, "Error: no se encontro %s\n", ruta); return 0; }
memset(n->mapa, '.', sizeof(n->mapa));
char linea[COLS + 4];
int fila = 0;
while (fila < FILAS && fgets(linea, sizeof(linea), f)) {
for (int col = 0; col < COLS && linea[col] && linea[col] != '\n'; col++)
n->mapa[fila][col] = linea[col];
fila++;
}
fclose(f);

// NASM calcula totales – NO se escriben a mano
n->total_monedas = (int)contar_caracter((char*)n->mapa, FILAS*COLS, 'M');
n->total_libres = (int)contar_libres ((char*)n->mapa, FILAS*COLS);
return 1;
}

void encontrar_jugador(Nivel *n, int *fila_out, int *col_out) {
for (int i = 0; i < FILAS; i++)
for (int j = 0; j < COLS; j++)
if (n->mapa[i][j] == 'P') { *fila_out = i; *col_out = j; return; }
*fila_out = 1; *col_out = 1; // fallback
}

```

include/objetos.h y src/objetos.c — base

objetos.c centraliza todos los efectos de recoger o interactuar con celdas. manejar_objeto() devuelve 1 si el jugador puede moverse a esa celda, 0 si no.

```

// include/objetos.h
#ifndef OBJETOS_H
#define OBJETOS_H
#include "mapas.h"
#include "jugador.h"

int manejar_objeto(Nivel *n, Jugador *j, int nf, int nc);
#endif // OBJETOS_H

// src/objetos.c
#include "../include/objetos.h"
#include "../include/rutinas.h"
#include "../include/assets.h"

int manejar_objeto(Nivel *n, Jugador *j, int nf, int nc) {
char celda = n->mapa[nf][nc];
if (celda == CHAR_PARED) return 0;
if (celda == CHAR_PUERTA && !j->tiene_llave) return 0;

```

```
if (celda == CHAR_MONEDA) { j->monedas++; n->mapa[nf][nc] = CHAR_LIBRE; }  
if (celda == CHAR_LLAVE) { j->tiene_llave=1; n->mapa[nf][nc] = CHAR_LIBRE; }  
// CHAR_SALIDA y CHAR_PUERTA: el jugador entra, main.c detecta la salida  
return 1;  
}
```

⇒ Entrega mapas.h y objetos.h el día 1 — desbloquea al Int. 1

⇒ Los 4 mapas .txt pueden ser simples el día 1 y refinarse el día 2

⇒ La música: colocar .wav libres de derechos en assets/ y usar las macros AUDIO_NIVELx

main.c — trabajo conjunto de los 3 integrantes

main.c conecta todo. No tiene lógica propia: solo carga niveles, llama funciones de los otros módulos y gestiona el bucle principal. Se construye en el día 3 cuando los demás módulos ya funcionan.

```
#include "../include/jugador.h"
#include "../include/renderizado.h"
#include "../include/mapas.h"
#include "../include/objetos.h"
#include "../include/rutinas.h"
#include "../include/assets.h"
#include // _getch() en Windows
#include // PlaySound

static const char *RUTAS_MAPA[NUM_NIVELES] = {
"mapas/nivel1.txt", "mapas/nivel2.txt",
"mapas/nivel3.txt", "mapas/nivel4.txt"
};

static const char *RUTAS_AUDIO[NUM_NIVELES] = {
AUDIO_NIVEL1, AUDIO_NIVEL2, AUDIO_NIVEL3, AUDIO_NIVEL4
};

int main(void) {
pantalla_inicio();
int mon_total=0, mon_max=0, pasos_total=0;
for (int i = 0; i < NUM_NIVELES; i++) {
Nivel n = {0};
n.num_nivel = i + 1;
if (!cargar_mapa(RUTAS_MAPA[i], &n;)) return 1;
// Musica del nivel (punto extra)
PlaySound(RUTAS_AUDIO[i], NULL, SND_FILENAME|SND_ASYNC|SND_LOOP);
int fi, ci;
encontrar_jugador(&n;, &fi;, &ci;);
Jugador j;
jugador_init(&j;, fi, ci);
n.mapa[fi][ci] = CHAR_LIBRE; // quitar la P del mapa
int nivel_terminado = 0;
while (!nivel_terminado) {
imprimir_ventana(&n;, &j;);
mostrar_hud(&j;, n.num_nivel);
char tecla = (char)_getch();
if (tecla=='q' || tecla=='Q') { PlaySound(NULL, NULL, SND_PURGE); return 0; }
mover_jugador(&n;, &j;, tecla);
// detectar si el jugador llego a la salida
if (detectar_objeto((char*)n.mapa, COLS, j.fila, j.col, CHAR_SALIDA))
nivel_terminado = 1;
}
PlaySound(NULL, NULL, SND_PURGE);
mostrar_resumen_nivel(&j;, &n;);
mon_total += j.monedas;
mon_max += n.total_monedas;
pasos_total+= j.pasos;
_getch();
}

long puntaje = calcular_puntaje(mon_total, pasos_total, NUM_NIVELES);
mostrar_resumen_final(mon_total, mon_max, pasos_total, puntaje);
pantalla_victoria(puntaje);
return 0;
```


Calendario — 4 días

Días 1–2: desarrollo paralelo intensivo. Día 3: integración + main.c. Día 4: pulido y entrega.

Día	Int. 1 — Jugador/Render	Int. 2 — NASM/Build	Int. 3 — Mapas/Objetos/Audio
Día 1 Paralelo	• jugador.h ← ENTREGAR • jugador_init() y mover_jugador() base • renderizado.h ← ENTREGAR • imprimir_ventana() con mapa hardcoded	• rutinas.h ← ENTREGAR • contar_caracter() en NASM • validar_movimiento() en NASM • build.bat base (sin mapas.c aún)	• mapas.h y objetos.h ← ENTREGAR • colores.h y assets.h completos • cargar_mapa() funcional • nivel1.txt y nivel2.txt (60×60)
Día 2 Paralelo	• mostrar_hud() • mostrar_resumen_nivel/final() • pantalla_inicio() y victoria() • lógica completa de mover_jugador	• detectar_objeto() en NASM • calcular_puntaje() en NASM • contar_libres() en NASM • build.bat completo + README.md	• manejar_objeto() completo • nivel3.txt y nivel4.txt • .wav en assets/ listos • encontrar_jugador()
Día 3 ■ Integración + main.c	• Conectar mapas reales a imprimir_ventana • Verificar flujo completo nivel 1→4 • Corregir bugs del renderizado	• build.bat final con -lwinmm • Prueba completa del build • Ajustar NASM si hay bugs	• main.c conjunto: bucle de niveles • Integrar PlaySound • Probar carga y flujo de 4 niveles
Día 4 Entrega	• Pulir colores, alineación • Commits finales del motor	• Verificar 5 funciones NASM • Último commit + historial GitHub	• Capturas de pantalla • Reporte PDF • Entrega en plataforma

Regla del día 1 — headers compartidos

Los tres integrantes entregan sus headers antes de escribir una línea de implementación. Con eso los demás pueden compilar usando stubs que devuelven 0.

```
// Stub temporal en jugador.c mientras NASM no esta listo:
// long validar_movimiento(char*m,long c,long f,long col){return 1;}
// long detectar_objeto(char*m,long c,long f,long col,char o){return 0;}
// Stub temporal en mapas.c mientras objetos.c no esta listo:
// int manejar_objeto(Nivel*n,Jugador*j,int f,int c){return 1;}
```

Puntos extra contemplados (+30)

Mejora	Pts	Responsable	Cómo
Colores ANSI	+5	Int. 3 (colores.h)	Ya definido en colores.h, Int. 1 lo usa en renderizado.c
Pantalla inicio + victoria	+5	Int. 1 (renderizado.c)	pantalla_inicio() y pantalla_victoria() en renderizado.c
Mapas desde archivos .txt	+5	Int. 3 (mapas.c)	cargar_mapa() con fopen/fgets — ya en la base
Más de 3 niveles (4 niveles)	+5	Int. 3 (nivel4.txt)	NUM_NIVELES=4 desde el inicio en mapas.h
Bloques ASCII	+5	Int. 3 (assets.h)	VISUAL_* de 2 chars ya definidos en assets.h
Música con PlaySound + .wav	+5	Int. 3 (assets.h) Int. 2 (build.bat -lwinmm) Todos (main.c)	AUDIO_NIVELx en assets.h, -lwinmm en build.bat, PlaySound() en main.c al iniciar cada nivel

TOTAL	+30		
-------	-----	--	--

Historial de commits sugerido

#	Mensaje de commit
1	Estructura de carpetas: src/ include/ mapas/ assets/ + archivos vacíos
2	Int3: colores.h, assets.h, mapas.h, objetos.h completos
3	Int2: rutinas.h con 5 prototipos + build.bat base
4	Int1: jugador.h, renderizado.h – structs y prototipos
5	Int3: cargar_mapa() + encontrar_jugador() + nivel1.txt y nivel2.txt
6	Int2: contar_caracter() y validar_movimiento() en NASM
7	Int1: jugador_init(), mover_jugador(), imprimir_ventana() base
8	Int3: manejar_objeto() completo + nivel3.txt y nivel4.txt + .wav en assets/
9	Int2: detectar_objeto(), calcular_puntaje(), contar_libres() + build.bat final -lwinmm
10	Int1: renderizado completo: hud, resúmenes, pantalla_inicio, pantalla_victoria
11	Integración: main.c conjunto + flujo nivel 1→4 probado
12	Correcciones, pulido visual y README completo